GENETIC ALGORITHM – QUICK REVISION SHEET

◆ 1. Chromosome Initialization

```
random.sample(range(n), n)
```

✓ Returns **random permutation (0 → n-1)**
✓ No duplicates
✓ Used for TSP / ordering problems

💡 *Memory*: "random permutation"

◆ 2. Fitness → Probability

```
probs = [f/total for f in fitnesses]
```

✓ Converts fitness → probabilities
✓ Sum of probs = **1**

Example:

```
[10,20,30] → [0.16, 0.33, 0.5]
```

💡 *Memory*: "fitness = score, probs = chance"

◆ 3. Roulette Wheel Selection

```
random.choices(chromosomes, weights=probs)[0]
```

✓ Selects **1 chromosome based on probability**
✓ Higher fitness → higher chance

⚠️ `weights=probs` is required
❌ NOT `key=probs`

◆ 4. Tournament Selection

```
selected = random.sample(list(zip(chromosomes, fitnesses)), k)
return max(selected, key=lambda x: x[1])[0]
```

✓ Pick **k random chromosomes**
✓ Return the **best (highest fitness)**

💡 *Memory*: "pick k → fight → best wins"

◆ 5. Single Point Crossover

```
child = p1[:point] + p2[point:]
```

✓ First part from p1
✓ Second part from p2

⚠ May create duplicates → NOT valid for TSP

◆ 6. Order Crossover (OX) ⭑ IMPORTANT

Steps:

1. Pick 2 points `a, b`
2. Copy segment from p1
3. Fill remaining from p2 (skip duplicates, maintain order)

✓ Produces **valid permutation**
✓ Preserves order + position

💡 *Memory*: "copy from p1, fill from p2 in order"

◆ 7. Mutation Operators

✅ (a) Scramble Mutation

```
subset = chromo[a:b]
random.shuffle(subset)
chromo[a:b] = subset
```

✓ Randomly shuffle a segment

💡 "mix randomly"

✅ (b) Inversion Mutation

```
chromo[a:b] = chromo[a:b][::-1]
```

✓ Reverse a segment

💡 “flip segment”

● reverse() issue

`chromo[a:b].reverse()` ✕

✓ Correct:

```
subset = chromo[a:b]
subset.reverse()
chromo[a:b] = subset
```

◆ 8. Why `.copy()` is used

```
inversion_mutation(chromosome.copy())
```

✓ Mutation changes list **in-place**
✓ `.copy()` protects original

💡 “copy = don’t destroy original”

◆ 9. Important Python Concepts

✓ Slicing

```
chromo[a:b]
```

- includes `a`
- excludes `b`

✓ Reverse

```
[::-1]
```

- quick reverse (preferred)

✓ zip()

```
list(zip(chromosomes, fitnesses))
```

→ `[('A',10), ('B',20)]`

✓ key usage

```
max(list, key=lambda x: x[1])
```

✓ compare using fitness

🔥 10. Common Mistakes (VERY IMPORTANT)

❌ Using `random.choice()` instead of `choices()`
❌ Writing `key=probs`
❌ Forgetting to skip duplicates in crossover
❌ Using `.reverse()` directly on slice
❌ Not using `.copy()`
❌ Breaking order in OX

⚔□ 11. Selection Comparison

Method	Uses Probability	Idea
Roulette	✅	fitness-based chance
Tournament	❌	best out of k

⚔□ 12. Mutation Comparison

Type	Behavior
Swap	swap 2 elements
Scramble	random shuffle
Inversion	reverse

🔥 FINAL FLOW (VERY IMPORTANT)

```
Initialize population
→ Calculate fitness
→ Convert to probabilities
→ Selection (roulette / tournament)
→ Crossover (OX)
→ Mutation (scramble / inversion)
→ Repeat
```

□ ULTRA SHORT MEMORY BLOCK

- sample → permutation
- probs → normalized fitness
- weights → selection chance
- tournament → best of k
- OX → copy + fill (no duplicates)
- scramble → shuffle
- inversion → reverse
- copy() → protect original